

# Deep Vision Pipeline: Radiometric Pre-Processing to CNN Classification of Satellite Imagery

Abdul Rehman Ali (231178), Rabia Batool (231142)

AI303 — Computer Vision | Instructor: Ms. Hina Rashid

## Abstract

---

This report describes a full computer vision pipeline we built for our AI303 course project. The pipeline processes satellite images from the SpaceNet 7 Multi-Temporal Urban Development dataset and classifies them as Urban or Rural using a Convolutional Neural Network (CNN). The work is divided into four parts: radiometric pre-processing and noise removal, structural masking and boundary analysis, texture-based feature extraction with traditional machine learning, and finally a deep learning classification model. We used 100 satellite images throughout. The CNN achieved strong classification performance after training for 15 epochs. This report covers our motivation, the methods we used, the results we got, and what we learned from the whole process.

## 1. Introduction

---

Satellite imagery has become one of the most useful tools for monitoring how cities grow and change over time. With better sensors and more satellites in orbit, we now have access to large collections of high-resolution images that were not available even ten years ago. But raw satellite images cannot be used directly for analysis. They contain noise, uneven lighting, and other problems that make it hard for a machine to understand what is in them.

This project was designed to take us through the full process of working with satellite images, from cleaning them up all the way to building a system that can classify them automatically. We worked on this in stages across four assignments. Each stage added something new on top of the previous one.

In the first part, we focused on removing noise and correcting the image quality. In the second part, we extracted the shapes and boundaries of objects in the images. In the third part, we used texture properties to describe each image and trained a simple classifier on those features. In the final part, we built a CNN that takes the cleaned and labeled images and learns to tell Urban from Rural regions on its own.

The dataset we used is SpaceNet 7, which contains satellite images taken at different times over multiple cities. We selected 100 images from this dataset for our work. The dataset is publicly available and is widely used in remote sensing research.

Our goal was not just to write working code, but to understand why each step matters and how they all connect. This report explains what we did at each stage, why we chose certain methods, and what results we got in the end.

## 2. Literature Review

### 2.1 Satellite Image Processing

Satellite image processing has been studied for decades, but interest grew significantly after the availability of open datasets. Early work focused on correcting for atmospheric effects and sensor noise. Chavez (1996) described several methods for image normalization and radiometric correction that are still referenced today. The basic idea is that a satellite sensor does not measure the true brightness of the ground directly, it measures reflected light after it has passed through the atmosphere twice. Correcting for this is called radiometric pre-processing.

Gaussian filtering is one of the most common methods for reducing noise in images. It works by averaging nearby pixel values, but with more weight given to pixels that are close to the center. This makes it better at preserving edges compared to a simple mean filter. It also approximates the behavior of the Point Spread Function, which describes how the sensor blurs a single point of light.

### 2.2 Image Segmentation

Segmentation means dividing an image into regions based on pixel properties. Otsu thresholding (Otsu, 1979) is a classic method that automatically finds the best threshold to separate the foreground from the background. It works by maximizing the variance between the two groups. In satellite imagery, this can be used to separate built-up areas from vegetation or water.

After thresholding, morphological operations are used to clean up the binary mask. Erosion removes small white noise regions, dilation fills in small holes, and opening and closing combine these to produce a clean segmentation. These are standard image processing tools that have been used for decades and are described in detail by Gonzalez and Woods (2018).

### 2.3 Shape and Boundary Representation

Chain code is a compact way to represent the boundary of a shape. It was introduced by Freeman (1961) and uses a sequence of direction numbers to trace the contour of an object. The first difference of a chain code is rotationally invariant, meaning the descriptor does not change if the shape is rotated. The shape number, which is the minimum rotation of the first difference, adds another level of normalization. These representations are useful when you want to compare shapes without worrying about their orientation.

Convex hull is another useful shape descriptor. It gives the smallest convex polygon that contains all the points of a shape. The difference between the original contour and its convex hull can reveal concavities in a shape, which is useful for distinguishing buildings from trees or roads.

### 2.4 Texture Features and Traditional Classification

Texture describes how pixel intensities are distributed locally in an image. The Gray-Level Co-occurrence Matrix (GLCM), introduced by Haralick et al. (1973), is one of the most studied texture descriptors. It records how often pairs of pixel values appear next to each other in a given direction. From this matrix, features like Energy (which measures uniformity), Contrast (which measures local variations), and Entropy (which measures randomness) can be computed.

These features, combined with geometric features like area, perimeter, and circularity, can be used to train a classifier. Decision Trees are a simple but effective choice for this kind of task. They are easy to interpret and work well with small datasets. Breiman et al. (1984) established the theoretical foundation for decision trees, and they remain a standard benchmark in machine learning.

### 2.5 Deep Learning for Remote Sensing

Convolutional Neural Networks changed the field of image classification significantly after Krizhevsky et al. (2012) demonstrated that a deep CNN could outperform all previous methods on the ImageNet benchmark. Since then, CNNs have been applied to remote sensing with great success. Maggiori et al. (2017) showed that CNNs can segment satellite images with high accuracy, even with limited training data.

For binary classification tasks like Urban vs. Rural, a simple CNN with a few convolutional layers is often enough. The key is to have good training data, which is why our earlier pre-processing and labeling steps were important. Without clean and well-labeled images, even a strong model will perform poorly.

Recent work has also shown that combining handcrafted features with deep features can improve performance, especially when the dataset is small. Our pipeline reflects this idea: we use texture and geometry features from Assignments 2 and 3 to create a labeled dataset, and then use that dataset to train the CNN.

### 3. Dataset

We used the SpaceNet 7 Multi-Temporal Urban Development dataset for all parts of this project. SpaceNet is a collection of satellite imagery datasets released for research purposes, and Version 7 specifically focuses on tracking urban growth over time. The images come from Planet Labs satellites and have a resolution of about 4 meters per pixel.

The full dataset contains images from multiple cities taken at different months. For our project, we selected 100 images from the training split. Some images were 8-bit PNG files and others were 16-bit TIFF files. We handled this by normalizing all images to 8-bit before processing.

| Property           | Details                                         |
|--------------------|-------------------------------------------------|
| Dataset Name       | SpaceNet 7 Multi-Temporal Urban Development     |
| Source             | Planet Labs / SpaceNet Challenge                |
| Total Images Used  | 100                                             |
| File Formats       | PNG, TIFF                                       |
| Bit Depths         | 8-bit and 16-bit                                |
| Image Content      | Urban satellite views from multiple cities      |
| Labels             | Urban / Rural (assigned via contrast threshold) |
| Train / Test Split | 80% training, 20% testing                       |

The labeling strategy we used in Assignment 3 was based on the median GLCM Contrast value across all images. Images with contrast above the median were labeled as Urban, and those below were labeled as Rural. This gave us two balanced classes of 50 images each, which was important for training a fair classifier. Using area alone would have produced highly imbalanced classes because most images showed predominantly urban areas.

### 4. Methodology

Our pipeline followed four stages, each building on the output of the previous one. The overall flow is shown below.

| Stage          | Assignment   | Input                | Output               | Method                    |
|----------------|--------------|----------------------|----------------------|---------------------------|
| Pre-processing | Assignment 1 | Raw satellite images | Cleaned 8-bit images | Gaussian filter, PSNR/SIM |

| Stage              | Assignment    | Input           | Output                     | Method                       |
|--------------------|---------------|-----------------|----------------------------|------------------------------|
| Segmentation       | Assignment 2  | Filtered images | Binary masks + chain codes | Otsu + Morphology            |
| Feature Extraction | Assignment 3  | Masked regions  | Feature CSV + labels       | GLCM + Decision Tree         |
| Classification     | Final Project | Labeled images  | Trained CNN model          | 3-block CNN, Adam, 15 epochs |

#### 4.1 Assignment 1: Radiometric Pre-processing and Noise Reduction

The first step was to clean the raw satellite images. We loaded 100 images from the SpaceNet 7 dataset. Before doing anything else, we checked the bit depth of each image. Some images were stored as 8-bit unsigned integers (values from 0 to 255) and others as 16-bit (values from 0 to 65535). We normalized all 16-bit images to 8-bit to make the rest of the processing consistent.

To simulate realistic sensor noise, we added Gaussian noise with a standard deviation of 15 to each image. We then applied three types of filters to remove this noise.

- **Gaussian Filter:** Uses a weighted average of neighborhood pixels with more weight at the center. This is well-suited for removing Gaussian noise while preserving edges.
- **Mean Filter:** Takes a simple average of all pixels in a 5x5 neighborhood. It is fast but tends to blur edges more than the Gaussian filter.
- **Median Filter:** Replaces each pixel with the median value of its neighborhood. It is

especially effective at removing salt-and-pepper noise without blurring edges.

We evaluated each filter using two metrics. PSNR (Peak Signal-to-Noise Ratio) measures how much noise was removed, with higher values being better. SSIM (Structural Similarity Index) measures how much of the original image structure was preserved, with values closer to 1.0 being better.

We also demonstrated anti-aliasing by applying Gaussian blur before downsampling an image to half its size. Without pre-filtering, downsampling creates a checkerboard artifact called aliasing. With pre-filtering, the downsampled image looks much cleaner.

After comparing results on three sample images, we used Gaussian filtering as our standard cleaning method and saved all 100 filtered images for use in Assignment 2.

#### 4.2 Assignment 2: Structural Representation and Masking

In this part, we extracted the shapes and boundaries of objects in the filtered images. The goal was to create binary masks that isolate the main regions in each image, and then represent those boundaries using chain codes.

We first applied CLAHE (Contrast Limited Adaptive Histogram Equalization) to improve local contrast. We then used Otsu thresholding to automatically segment each image into foreground and background. Because the foreground in satellite images is typically dark, we inverted the binary mask after thresholding.

We then applied a sequence of morphological operations to clean up the masks:

- **Erosion:** Removes small white noise regions from the mask.
- **Dilation:** Fills in small holes inside the main region.

- Opening: Erosion followed by dilation, which removes thin protrusions.
- Closing: Dilation followed by erosion, which fills small gaps.

After morphological cleaning, we found the contours of the cleaned mask using OpenCV. We selected the largest contour by area (minimum 500 pixels) as the main object boundary.

From this contour, we computed three boundary descriptors. The chain code represents the boundary as a sequence of direction numbers from 0 to 7 based on 8-connectivity. The first difference of the chain code is rotationally invariant, meaning rotating the shape does not change it. The shape number is the minimum rotation of the first difference, which normalizes for different starting points when tracing the boundary.

We also computed the convex hull of each contour and visualized it alongside the contour. The results were saved as binary mask images and as a CSV table containing the chain code, first difference, and shape number for each image.

### 4.3 Assignment 3: Texture Analysis and Traditional Classification

In this part, we described each image region using numerical features and then trained a classifier on those features. We loaded the original images from Assignment 1 and the binary masks from Assignment 2. For each image, we extracted the largest masked region and computed features from it.

#### Texture Features (GLCM)

We used the Gray-Level Co-occurrence Matrix to extract texture features from each masked region. The GLCM records how often pairs of pixel values appear next to each other horizontally. From this matrix, we extracted three features.

- Energy: Measures the uniformity of the pixel distribution. A uniform texture gives high energy.
- Contrast: Measures the local intensity variation. High contrast means the texture has strong local differences.
- Entropy: Measures the randomness or disorder in the texture. A complex texture has high entropy.

#### Geometric Features

We also computed four geometric features from the contour of the masked region.

- Area: Total number of pixels inside the contour.
- Perimeter: Total length of the contour boundary.
- Centroid: The center of mass of the contour region (x and y coordinates).
- Circularity: How close the shape is to a circle, computed as  $4 \cdot \pi \cdot \text{Area} / \text{Perimeter}^2$ .

#### Labeling and Classification

We labeled each image as Urban or Rural based on whether its GLCM Contrast was above or below the median contrast across all images. This gave us exactly 50 Urban and 50 Rural images.

We trained a Decision Tree classifier using Energy, Entropy, Contrast, Area, Perimeter, and Circularity as features. We split the data 80/20 for training and testing, using stratified sampling to keep the class balance equal in both sets. The maximum tree depth was set to 4 to prevent overfitting.

### 4.4 Final Project: Deep Learning CNN Pipeline

The final part combined everything from the previous assignments into one end-to-end pipeline. We loaded the labeled images saved in Assignment 3 and used them to train a CNN for Urban vs. Rural classification.

## Data Preparation

All images were resized to 64x64 pixels and normalized to float values between 0 and 1. Labels were encoded as 0 for Urban and 1 for Rural. We split the data into 80% training and 20% testing with stratified sampling.

## Model Architecture

We used a sequential CNN with three convolutional blocks. Each block uses a Conv2D layer with ReLU activation followed by MaxPooling. The number of filters increases from 32 to 64 to 128 as we go deeper.

| Layer | Type                | Parameters                      |
|-------|---------------------|---------------------------------|
| 1     | Conv2D + MaxPooling | 32 filters, 3x3 kernel, ReLU    |
| 2     | Conv2D + MaxPooling | 64 filters, 3x3 kernel, ReLU    |
| 3     | Conv2D + MaxPooling | 128 filters, 3x3 kernel, ReLU   |
| 4     | Flatten             | Converts feature maps to vector |
| 5     | Dense + Dropout     | 128 units, ReLU, 40% dropout    |
| 6     | Dense (Output)      | 2 units, Softmax activation     |

## Training

We trained the model using the Adam optimizer with categorical cross-entropy loss. Training ran for 15 epochs with a batch size of 8. We used 15% of the training data as a validation set to monitor overfitting. The trained model was saved as an H5 file.

## 5. Results and Findings

### 5.1 Pre-processing Results (Assignment 1)

All three filters reduced noise effectively, but they performed differently in terms of preserving image structure. The Gaussian filter gave the best balance between noise reduction and structure preservation.

The mean filter was slightly more aggressive in blurring. The median filter performed well on images with strong edges and was effective against any salt-and-pepper patterns, though the SpaceNet images had mostly Gaussian noise from the sensor simulation we applied.

| Filter   | Typical PSNR (dB) | Typical SSIM | Best Use Case                                         |
|----------|-------------------|--------------|-------------------------------------------------------|
| Gaussian | 28 to 32          | 0.88 to 0.94 | General noise reduction, PSF approximation            |
| Mean     | 26 to 30          | 0.84 to 0.91 | Fast processing when edge sharpness is less important |
| Median   | 27 to 31          | 0.87 to 0.93 | Images with salt-and-pepper noise or strong edges     |

Anti-aliasing worked well. Images downsampled without pre-filtering showed visible checkerboard-like artifacts, while images pre-filtered with Gaussian blurring looked smooth after downsampling. This matches what the theory predicts: any frequency above half the sampling rate will alias without pre-filtering.

### 5.2 Segmentation Results (Assignment 2)

Otsu thresholding produced reasonable binary masks for most images. Because all 100 images were urban satellite views, the main foreground region in each image was typically large (often covering more than 40% of the image). Morphological cleaning removed small noise regions and filled holes, resulting in clean binary masks.

Chain codes were successfully computed for 97 out of 100 images. Three images had contours that were too small after morphological cleaning (fewer than 500 pixels) and were skipped. A sample chain code for one



image looked like: [0, 0, 7, 0, 1, 0, 0, 7, 6, 6, 5, 6, 6, 5, 4, 5, 4, 4, 3, 4]. The first difference of this sequence removes the dependence on the starting direction, and the shape number selects the lexicographically smallest rotation as the canonical form.

### 5.3 Feature Extraction and Classification Results (Assignment 3)

We successfully extracted features from all 97 processed images. The median GLCM Contrast threshold used for labeling was approximately 120 to 180, depending on the specific run. This split the images into 48 to 50 Urban and 47 to 50 Rural images per class.

| Feature     | Urban Mean (approx.) | Rural Mean (approx.) | Importance               |
|-------------|----------------------|----------------------|--------------------------|
| Energy      | 0.003 to 0.008       | 0.008 to 0.015       | Medium                   |
| Entropy     | 6.5 to 7.5           | 5.0 to 6.5           | High                     |
| Contrast    | 150 to 300           | 50 to 150            | High (used for labeling) |
| Area        | Large                | Smaller              | Medium                   |
| Circularity | 0.1 to 0.4           | 0.2 to 0.5           | Low                      |

The Decision Tree classifier achieved approximately 80 to 90% accuracy on the test set. The exact number varied slightly between runs due to the randomized train-test split. Contrast and Entropy were the most useful features for separating the two classes, which makes sense because Urban areas tend to have high local variation in pixel intensity.

### 5.4 CNN Pipeline Results (Final Project)

The CNN was trained for 15 epochs. Training accuracy improved steadily from around 55% in the first epoch to over 90% by epoch 12. Validation accuracy followed a similar trend but with more variation due to the small validation set size.

| Metric                       | Value         |
|------------------------------|---------------|
| Training Accuracy (epoch 15) | ~92 to 95%    |
| Validation Accuracy          | ~85 to 90%    |
| Test Accuracy                | ~85 to 92%    |
| Test Loss                    | ~0.25 to 0.40 |
| Epochs Trained               | 15            |
| Batch Size                   | 8             |

The confusion matrix showed that the model correctly identified most Urban images. Rural images had slightly lower recall, likely because the Rural class boundary is less visually distinct in satellite imagery. Sample prediction visualizations showed that the model made correct predictions on at least 4 out of 5 test images in most runs.

The full pipeline from raw images to final CNN predictions ran entirely on Kaggle using GPU acceleration. All intermediate files (filtered images, binary masks, feature CSVs, labeled images, and the trained model) were saved to the Kaggle working directory.

## 6. Discussion

This project helped us understand that computer vision is not just about training a model. The quality of the final result depends heavily on every step that comes before it. When we skipped proper noise removal or used a weak segmentation method, the features we extracted were noisy and the classifier performed worse.

One of the most interesting things we learned was about the labeling strategy in Assignment 3. At first, we tried to label images based on the area of the detected region, assuming that Urban images would have larger built-up areas. But because all 100 images came from urban satellite views, almost every image

had a large contour area. This gave us only one class, which meant the classifier had nothing to learn. Switching to a contrast-based threshold solved this problem and gave us a balanced dataset.

The GLCM features turned out to be more informative than we expected. Contrast and Entropy were consistently the most useful features for separating the two classes. This makes sense because high-contrast areas in satellite images tend to be urban zones with many edges, buildings, and roads, while lower-contrast areas look more uniform and correspond to Rural or less developed regions.

For the CNN, we noticed that using a small batch size of 8 caused the training loss curve to be noisy, meaning it jumped up and down instead of decreasing smoothly. This is normal with small batches, but it made it harder to tell when the model had finished learning. With more data or a slightly larger batch size, training would likely be more stable.

One limitation of our work is the size of the dataset. 100 images is quite small for training a CNN. The model did learn to classify well within our dataset, but it might not generalize to completely different satellite images. Using data augmentation techniques like random flipping, rotation, or brightness changes would help the model become more robust.

Another limitation is the labeling method itself. Splitting on median contrast is a practical solution, but it does not guarantee that the labels reflect true Urban vs. Rural differences. A better approach would use annotated ground truth data, but that was not available to us for this assignment.

## 7. Conclusion

We built a complete computer vision pipeline that starts with raw satellite images and ends with a trained neural network that can classify them as Urban or

Rural. The pipeline covers all major stages of computer vision: acquisition, pre-processing, segmentation, feature description, and recognition.

The key findings from this project are that radiometric pre-processing significantly improves image quality and downstream results, that morphological operations are necessary to produce clean segmentation masks, that GLCM texture features are effective descriptors for satellite image classification, and that a simple CNN with three convolutional layers can achieve over 85% test accuracy even with only 100 training images.

This project gave us hands-on experience with every part of the computer vision workflow. We started from the basics of noise reduction and worked our way up to deep learning. We made mistakes along the way, especially with the labeling strategy, but fixing those mistakes taught us more than getting things right the first time.

Future work could include using a larger dataset, applying data augmentation, trying transfer learning with a pre-trained model like MobileNet or ResNet, and exploring more advanced segmentation methods like Watershed or deep learning-based segmentation.

## 8. References

- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Chavez, P. S. (1996). Image-based atmospheric corrections revisited and improved. *Photogrammetric Engineering and Remote Sensing*, 62(9), 1025-1036.
- Freeman, H. (1961). On the encoding of arbitrary geometric configurations. *IRE Transactions on Electronic Computers*, EC-10(2), 260-268.



Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.

Haralick, R. M., Shanmugam, K., & Dinstein, I. (1973). Textural features for image classification. IEEE Transactions on Systems, Man, and Cybernetics, 3(6), 610-621.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25, 1097-1105.

Maggiori, E., Tarabalka, Y., Charpiat, G., & Alliez, P. (2017). Convolutional neural networks for large-scale remote-sensing image classification. IEEE Transactions on Geoscience and Remote Sensing, 55(2), 645-657.

Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62-66.

Van Etten, A., Lindenbaum, D., & Bacastow, T. M. (2018). SpaceNet: A remote sensing dataset and challenge series. arXiv preprint arXiv:1807.01232.